

Software Design Specifications

(Mujawwad)

Project Code

TAJWEED-AI-22-26

Internal Advisor

Dr. Hussam Ali (internal)

External Advisor

Mr. Bilal Arshad (external)

Project Manager

Prof. Dr. Muhammad Ilyas

Project Team

Muhammad Haris (BSCS51F22S065) - Team Lead
Ahmad Raza Gul (BSCS51F22S054) - Team Member
Mahtab Ahmad (BSCS51F22S048) - Team Member

Submission Date

December 15, 2025

Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	Mujawwad
Document	Software Design Specification
Document Version	1.0
Identifier	TAJWEED-AI-22-26
Status	Under-review
Author(s)	Muhammad Haris, Ahmad Raza Gul, Mahtab Ahmad
Approver(s)	PM
Issue Date	December 15, 2025
Document Location	
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
ASP	Active Server Pages
DD	Design Specification
ASR	Automatic Speech Recognition
DTW	Dynamic Time Warping
API	Application Programming Interface
JSON	JavaScript Object Notation
REST	Representational State Transfer
JWT	JSON Web Token
GPU	Graphics Processing Unit
AI	Artificial Intelligence

Contents

1	Introduction	3
1.1	Purpose of Document	3
1.2	Project Overview	3
1.3	Scope	3
2	Design Considerations	4
2.1	Assumptions and Dependencies	4
2.2	Dependencies	5
2.3	Risks and Volatile Areas	5
3	System Architecture	7
3.1	System Level Architecture	7
3.2	Sub-System / Component / Module Level Architecture	8
3.3	Sub-Component / Sub-Module Level Architecture	9
4	Design Strategies	11
4.1	Strategy 1: Asynchronous Processing for AI Inference	11
4.2	Strategy 2: Modular Plug-and-Play AI Architecture	11
4.3	Strategy 3: Stateless Backend for Scalability	11
4.4	Strategy 4: Simple User Interface Paradigm	12
4.5	Strategy 5: Centralized Server-Side Processing	12
4.6	Strategy 6: Framework Selection - Django REST Framework	12
4.7	Strategy 7: Structured Data Management (SQL over NoSQL)	12
4.8	Strategy 8: Client-Side Downsampling	13
4.9	Strategy 9: Monitoring and Response Strategy	13
5	Detailed System Design	14
5.1	Use Case Diagram	14
5.2	Class Diagram	15
5.3	Sequence Diagram	17
5.4	Logical E/R Diagram	18
5.5	State Machine Diagram	20
5.6	Graphical User Interface	23
	References	24

Introduction

1.1 Purpose of Document

The purpose of this document is to lay out the detailed and complete architectural design for Mujawwad. It is intended for the project development team to guide the implementation phase and for the project supervisors for evaluation purposes. We utilize Object-Oriented Analysis and Design (OOAD) to decompose the system into modular classes and objects.

1.2 Project Overview

Mujawwad is a software system designed to assist users in improving their Quran recitation. Mistaking tajweed changes the meaning of the word; our system aspires to mitigate this issue. The system accepts audio input from the user, processes the speech signal using Signal Processing, machine learning and specific algorithms for pattern matching, detects pronunciation errors based on tajweed rules, and provides immediate feedback. The design prioritizes accuracy, modularity and ease of use for the users of the system.

1.3 Scope

1.3.1 In Scope

1. Phoneme-level recitation verification with error detection
2. Detection and feedback for Tajweed errors: Madd, Ghunnah, Qalqalah, and Noon/Meem Sakin rules
3. Visual highlighting of error locations within verse text
4. Support for Hafs recitation style
5. Support for Naskh script
6. User interface for verse selection and audio recording

1.3.2 Out of Scope

1. Automatic verse identification from audio (users must manually select the verse)
2. Multi-dialect or multi-Qiraat support
3. Advanced Tajweed rules beyond the specified core set
4. Quran search or browsing features
5. Social or competitive features

Design Considerations

This section describes the issues which need to be addressed or resolved before attempting to devise a complete design solution.

2.1 Assumptions and Dependencies

2.1.1 Noise Environment

The design assumes that the user will record audio in a moderately quiet environment. The current architectural approach does not include a dedicated Source Separation or Active Noise Cancellation module; it assumes the Signal-to-Noise Ratio (SNR) will be sufficient for the AI model to distinguish phonemes.

2.1.2 Recitation Style

The design assumes the recitation follows the Hafs'an Asim reading style. The algorithmic logic for error detection is currently specific for this style and may flag valid readings from other styles (like Warsh) as errors.

2.1.3 Connectivity Strategy

The design assumes a Client-Server architecture where the user has a stable internet connection. We are assuming the mobile device does not have sufficient processing power to run the full Tajweed analysis model locally (Edge AI), so the design relies on sending audio blobs to the server.

2.1.4 Input Format Standardization

We assume that regardless of the user's hardware, the client-side application can normalize audio input to a standard single-channel (mono), 16kHz WAV format before processing.

2.1.5 Hardware Requirements

- User devices possess microphone capabilities with a minimum sampling rate of 16kHz for adequate audio quality

2.1.6 Model Performance

- Trained models achieve minimum 75% accuracy in Tajweed rule detection.
- Audio processing pipeline maintains consistency across different reciter accents and styles.

2.1.7 Network Connectivity

- Users have stable internet connection for server communication.

- Network bandwidth supports audio streaming.

2.1.8 User Knowledge

- Users have basic understanding of Arabic script and Quranic text
- Users can read Quranic verses even if pronunciation is imperfect

2.2 Dependencies

2.2.1 AI Model Framework Compatibility

The backend design is strictly dependent on the PyTorch framework and specific audio processing libraries (e.g., Librosa or Torchaudio). Any changes to the core operations in future versions of these libraries may require significant refactoring of the inference engine.

2.2.2 Pre-trained ASR Models

The system relies on the availability of pre-trained speech models like Wav2Vec 2.0 to generate initial phoneme embeddings. The architecture assumes these models remain accessible and compatible with the finetuning layers.

2.2.3 Cloud GPU Availability

The system architecture depends on the availability of GPU-accelerated compute instances (e.g., AWS EC2 with CUDA support) to handle real-time inference latency constraints.

2.2.4 Mobile Microphone Permissions

The mobile client design depends on the underlying Operating System granting high-priority access to the microphone hardware to capture raw audio at the required sample rate (e.g., 16kHz or higher).

2.3 Risks and Volatile Areas

2.3.1 Network Latency and Timeout Risks

Risk: Since the design relies on server-side processing, large audio files or slow client internet connections could lead to request timeouts or poor user experience (high latency). If the round-trip time (RTT) exceeds 3-5 seconds, the application may feel unresponsive.

Design Contingency: The system architecture will incorporate an Asynchronous Task Queue (e.g., Celery). Instead of blocking the HTTP request while the AI processes the audio, the server will immediately return a Processing status (ID) to the client. The client will then poll the server or listen via WebSockets for the result.

2.3.2 AI Model Accuracy Volatility

Risk: The Tajweed detection model is a probabilistic component. There is a risk of high False Positive rates (flagging correct recitation as wrong) or False Negatives. This is

a Volatile Area because the model will likely need frequent updates and retraining even after the initial deployment.

Design Contingency: The design will follow a Modular Microservice approach for the AI engine. The AI inference logic will be containerized separately (e.g., Docker) from the main application backend. This allows us to hot-swap the model version (e.g., upgrading from v1.0 to v1.1) without taking down the entire API.

2.3.3 Audio Quality Variance

Risk: Users will utilize a wide variety of mobile microphones (low-end vs. high-end devices). The current design might fail if the input audio has significant background noise or clipping, which causes the phoneme recognition to degrade.

Design Contingency: A robust Audio Pre-processing Layer will be designed as the entry point of the pipeline. This module will normalize volume (Gain Control) and apply a Noise Suppression filter (using DSP techniques) before the audio is fed into the AI model.

2.3.4 Scalability and GPU Resource Contention

Risk: AI inference is computationally expensive. If the user base spikes unexpectedly, the server GPU may become a bottleneck, causing the service to crash or queue times to become unacceptable.

Design Contingency: The backend will be designed to be Stateless, allowing for Horizontal Scaling. We will design the system to sit behind a Load Balancer (e.g., Nginx), which can distribute traffic across multiple API instances.

2.3.5 Volatile Areas

- **Feature Extraction Pipeline:** May switch from MFCC to mel-spectrograms or phonemes based detection system.
- **Tajweed Rule Coverage:** Will incrementally add support for more Tajweed rules (start with 3-4, expand to 13+).
- **User Interface Requirements:** Feedback presentation may evolve based on user studies (text, audio, visual).
- **Backend Framework:** May need to switch from Django to FastAPI for better async performance

System Architecture

This section provides a high-level overview of how the functionality and responsibilities of the system are partitioned and then assigned to subsystems or components.

3.1 System Level Architecture

The AI Tajweed Recitation Checker follows a Client-Server Architecture (specifically a 3-Tier Layered Architecture). This approach decouples the resource-intensive AI processing from the client device, allowing the system to utilize powerful server-side GPUs for accurate phoneme analysis while maintaining a lightweight mobile application.

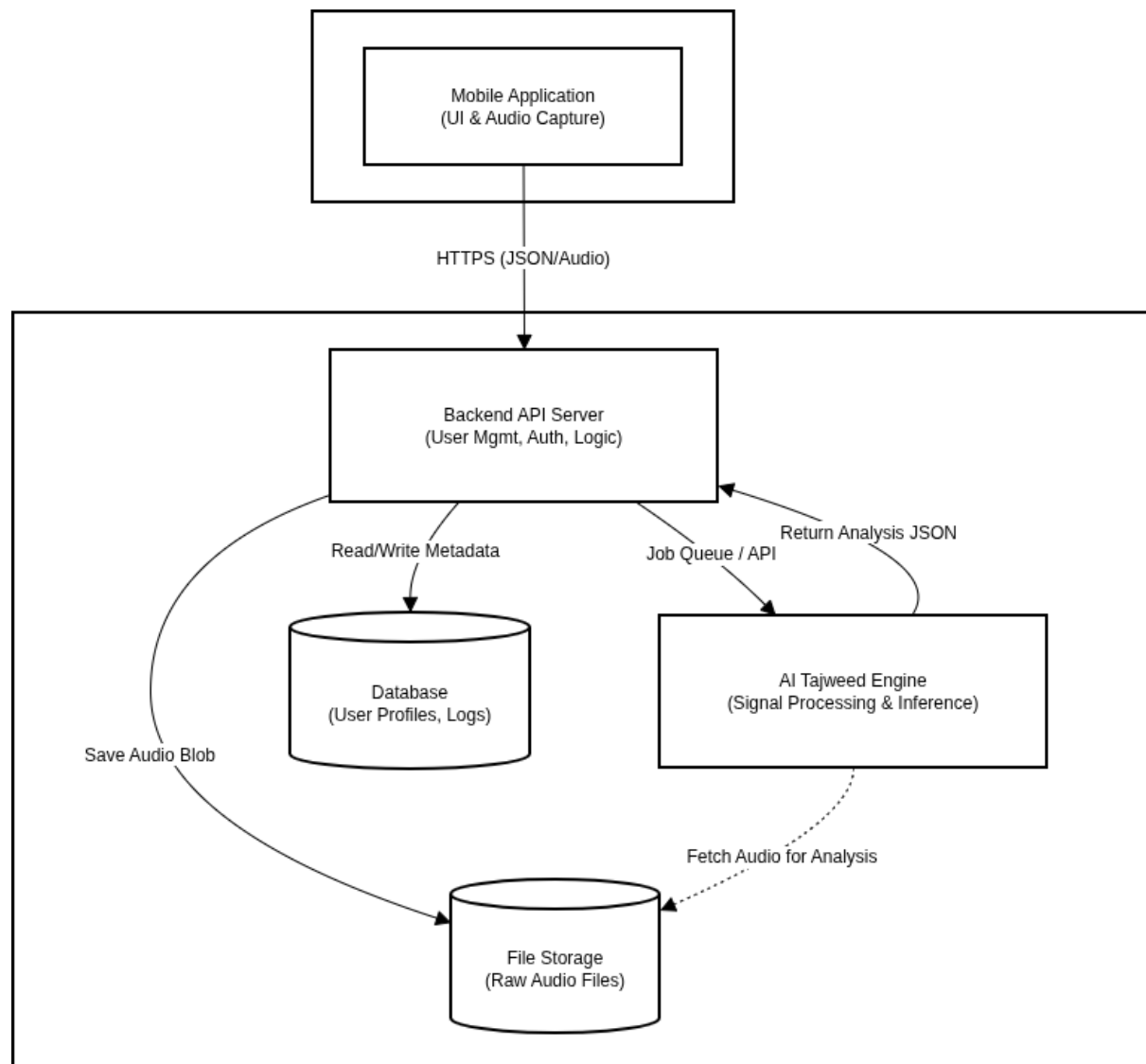


Figure 3.1: System Level Architecture of Mujawwad

The system is decomposed into four major high-level elements:

3.1.1 Presentation Layer (Mobile Client)

Role: Responsible for the User Interface (UI), capturing raw audio input from the user's microphone, and presenting the feedback (highlighted mistakes) received from the server.

Execution Environment: User's Mobile Device (Android/iOS).

3.1.2 Application Logic Layer (API Server)

Role: Acts as the central controller. It handles user authentication, receives audio uploads, manages session state, and orchestrates the communication between the database and the AI Engine.

Execution Environment: Web Server (e.g., Python Flask/Django or Node.js) hosted on the Cloud.

3.1.3 Intelligence Layer (AI Engine)

Role: A specialized module dedicated to signal processing. It accepts audio files, performs noise reduction, converts speech to phonemes, compares them against correct Tajweed rules, and generates an error report.

Execution Environment: GPU-accelerated Container (e.g., Docker running PyTorch/TensorFlow).

3.1.4 Data Layer (Storage)

Role: Persistent storage for user profiles, history logs, and the raw audio files.

Execution Environment: Relational Database (e.g., PostgreSQL) and File Object Storage (e.g., AWS S3 or Server File System).

3.2 Sub-System / Component / Module Level Architecture

The System Architecture is further decomposed into functional modules within the Intelligence Layer (AI Engine). This decomposition ensures a Separation of Concerns, isolating signal processing from business logic.

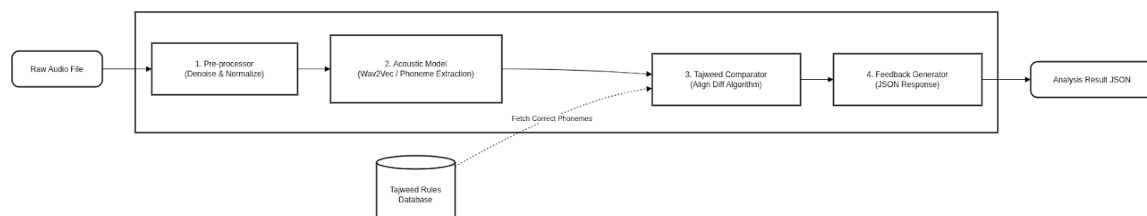


Figure 3.2: Tajweed Comparator Component Architecture

The AI Engine is divided into the following four core modules:

3.2.1 Audio Pre-processor

Function: Accepts the raw WAV file. It performs Noise Reduction (spectral gating), Silence Removal (trimming start/end silence), and Normalization (adjusting amplitude to -3dB) to prepare the audio for the model.

3.2.2 Acoustic Model

Function: This is the deep learning core (e.g., Wav2Vec 2.0). It takes the cleaned audio spectrogram and generates a sequence of Phoneme Probabilities (time-stamped phonetic transcriptions).

3.2.3 Tajweed Comparator

Function: This module contains the Ground Truth logic. It aligns the user's predicted phonemes with the correct Quranic phonemes (using algorithms like Dynamic Time Warping or Levenshtein Distance). It identifies specific deviations (e.g., a missing Ghunna or incorrect Madd).

3.2.4 Scoring and Feedback Generator

Function: Aggregates the errors found by the Comparator. It calculates a numerical accuracy score (0-100%) and formats the specific error messages into a JSON response for the client.

3.3 Sub-Component / Sub-Module Level Architecture

We will focus on the decomposition of the Tajweed Comparator Module, which is the core decision-making unit within the AI Engine.

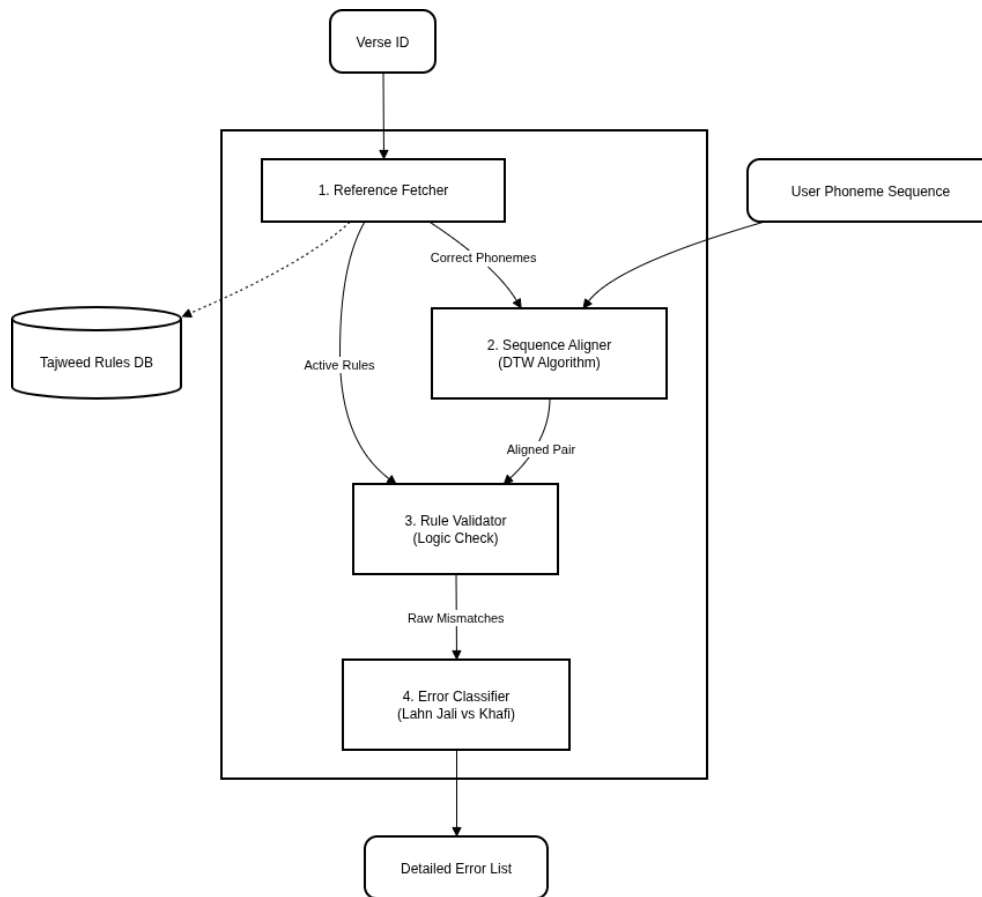


Figure 3.3: Sub-Component Diagram

The Tajweed Comparator is further divided into four interacting sub-components:

3.3.1 Reference Fetcher

Role: Based on the Verse ID identified or selected, this component retrieves the Ground Truth data (correct phoneme sequence and active Tajweed rules) from the Rules Database.

3.3.2 Sequence Aligner

Role: It uses a Dynamic Time Warping (DTW) or Needleman-Wunsch algorithm to align the user's spoken phoneme sequence with the correct reference sequence, handling speed variations or skipped sounds.

3.3.3 Rule Validator

Role: Once aligned, this component checks specific positions for rule compliance (e.g., Did the user hold the Ghunna for 2 beats? Was the Qalqalah bounced?).

3.3.4 Error Classifier

Role: Categorizes any detected mismatches into Major Errors (Lahn Jali) (changing meaning/letter) or Minor Errors (Lahn Khafi) (imperfect beauty/timing).

Design Strategies

This section outlines the key architectural strategies adopted to ensure the system is scalable, maintainable, and user-friendly.

4.1 Strategy 1: Asynchronous Processing for AI Inference

Description: The system utilizes an asynchronous task queue (e.g., Celery/Redis) to handle audio analysis. When a user uploads a recitation, the API immediately returns a Job ID (HTTP 202 Accepted) instead of waiting for the AI to finish. The client then polls for the result.

Reasoning: Deep learning inference (ASR and Phoneme alignment) is computationally expensive and can take several seconds to process. Keeping a standard HTTP connection open for this duration would lead to timeouts and a poor user experience.

Trade-off: This adds complexity to the client-side logic (which must implement polling or WebSocket listeners) but ensures the server remains responsive under load.

4.2 Strategy 2: Modular Plug-and-Play AI Architecture

Description: The Intelligence Layer is designed as an independent microservice container. The core application logic communicates with the AI model via a strict internal API contract.

Reasoning: AI research evolves rapidly. By isolating the AI engine, we can upgrade the underlying model (e.g., switching from Wav2Vec 2.0 to Whisper) without rewriting the backend code or the mobile app.

Trade-off: There is a slight latency overhead due to inter-service communication (serialization/deserialization of data) compared to a monolithic design.

4.3 Strategy 3: Stateless Backend for Scalability

Description: The backend API is designed to be stateless. No user session data is stored in the local server memory; instead, all session states are managed via external tokens (JWT [3]) and a shared database.

Reasoning: This allows the system to scale horizontally. If user traffic spikes, we can spin up multiple instances of the API server behind a load balancer without worrying about sticky sessions.

Trade-off: Every request must carry authentication tokens, slightly increasing bandwidth usage, but this is negligible compared to the benefits of reliability.

4.4 Strategy 4: Simple User Interface Paradigm

Description: The mobile UI follows a Record-Review-Feedback loop. The interface is minimal, focusing on a large microphone button and clear, color-coded text for feedback (e.g., Red for errors, Green for correct).

Reasoning: The target audience may range from children to elderly users. A complex interface would distract from the spiritual and educational purpose of Quran recitation.

Trade-off: Advanced features (like detailed spectrogram analysis) are hidden or omitted to maintain simplicity.

4.5 Strategy 5: Centralized Server-Side Processing

Description: All audio analysis and AI inference tasks are offloaded to the cloud server. The mobile client acts as a thin client, responsible only for capturing audio and displaying results.

Reasoning: The Tajweed analysis models require substantial VRAM and GPU compute capability, which exceeds the hardware specifications of standard mobile devices. Centralizing processing ensures consistent accuracy and performance.

Trade-off: This creates a strict dependency on internet connectivity. Users cannot utilize the core checking feature while offline.

4.6 Strategy 6: Framework Selection - Django REST Framework

Description: The backend Application Logic Layer is built using Django REST Framework (DRF) [2] rather than micro-frameworks like Flask or FastAPI.

Reasoning: DRF provides a robust, batteries-included architecture with built-in support for ORM (Object-Relational Mapping), authentication, and serialization. This accelerates development by reducing boilerplate code.

Trade-off: Django is heavier and has a steeper learning curve than Flask. It may introduce slightly higher overhead for very simple requests.

4.7 Strategy 7: Structured Data Management (SQL over NoSQL)

Description: The system utilizes a Relational Database Management System (PostgreSQL) instead of a NoSQL solution (like MongoDB).

Reasoning: The data model for this application is highly structured and relational. A User has many Recitations; a Recitation belongs to a specific Surah and Verse. SQL ensures referential integrity and supports complex joins required for analytics.

Trade-off: Scaling a relational database horizontally (sharding) is more complex than scaling a NoSQL store, but the expected data volume and structure prioritize consistency.

4.8 Strategy 8: Client-Side Downsampling

Description: The mobile application performs immediate downsampling of the raw audio recording from the device's native sampling rate (typically 44.1kHz or 48kHz) to 16kHz before uploading it to the server.

Reasoning:

- **Nyquist-Shannon Sampling Theorem** [1]: Human speech frequencies generally fall below 8kHz. A sampling rate of 16kHz is sufficient to reconstruct the signal without aliasing.
- **Bandwidth Efficiency:** Downsampling reduces the file size by approximately 3x. This significantly lowers network latency during upload and reduces storage costs.
- **Model Compatibility:** Most ASR models are pre-trained on 16kHz audio. Handling this conversion on the client side saves the server from wasting CPU cycles on transcoding.

Trade-off: This process is lossy; frequencies above 8kHz are discarded. While these frequencies usually contain environmental noise rather than speech information, there is a minor risk of losing high-frequency harmonics.

4.9 Strategy 9: Monitoring and Response Strategy

Description: Implement comprehensive logging and monitoring (error rates, latency metrics, model accuracy).

Detailed System Design

This section contains the detailed architectural diagrams illustrating the system's design from multiple perspectives.

5.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between users (Learner and Admin) and the various system functionalities including login, verse selection, audio recording, and Tajweed analysis.

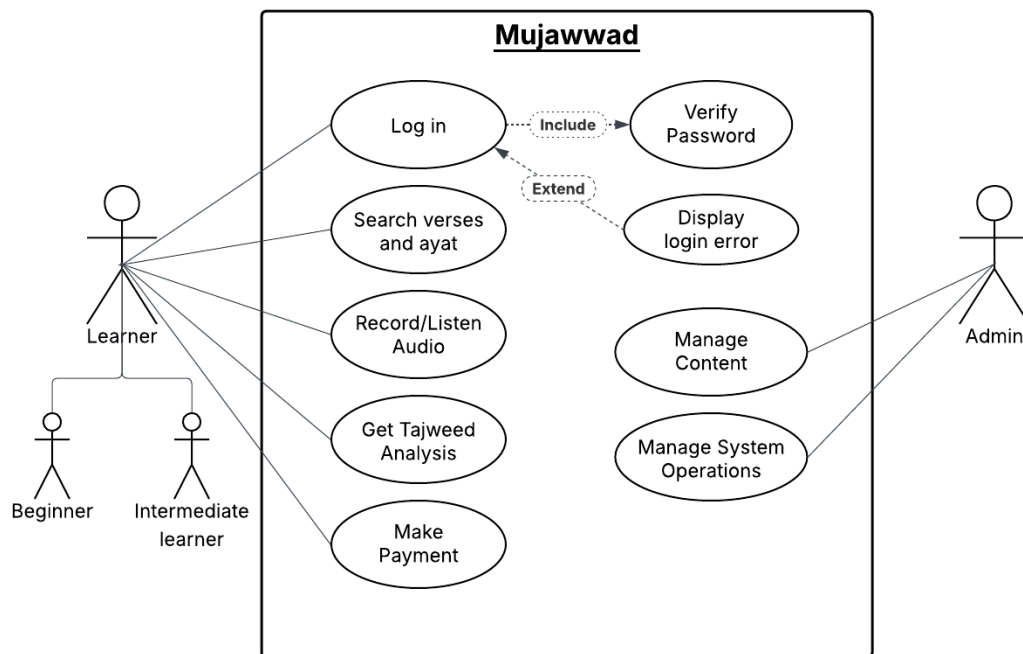


Figure 5.1: Use Case Diagram

5.1.1 Use Case Diagram Explanation

This diagram defines the functional requirements from a user-centric perspective, showing who can perform which actions within the Mujawwad system.

5.1.1.1 Actors

Learner (Primary Actor): The main user who practices Quran recitation. The Learner actor has two specializations shown with inheritance arrows: Beginner (users new to Tajweed rules) and Intermediate learner (users with some prior knowledge). Both specializations inherit all use cases available to the parent Learner actor but may experience different UI complexity or hints.

Admin (Secondary Actor): System administrator responsible for content management and system maintenance. The Admin has access to privileged operations not available to regular learners.

5.1.1.2 Learner Use Cases

Log in: The entry point for authentication. This use case "Verify Password," is a mandatory step that always executes during login—you cannot log in without it. The dependency is unidirectional, Log in depends on Verify Password.

Select verses and ayat: Allows authenticated users to find specific Quranic passages by selecting Surah number and Ayah number in the Arabic text. This is a standalone use case with no dependencies, enabling users to select the Ayah before deciding what to recite.

Record/Listen Audio: Core functionality enabling users to capture their recitation using the device microphone and playback previously recorded audio. This use case does not show explicit dependencies but implicitly requires prior verse selection.

Get Tajweed Analysis: The AI-powered evaluation feature that analyzes recorded audio and returns error feedback. This use case depends on "Record/Listen Audio" completing successfully, though the diagram does not show this dependency explicitly—it is implied through the workflow sequence.

Make Payment: Enables users to subscribe to premium features (e.g., advanced Tajweed rules, detailed analytics). After successful payment processing, the user gains access to restricted content.

5.1.1.3 Admin Use Cases

Manage Content: Allows administrators to add, edit, or delete Quranic text, reference audio files, and Tajweed rule definitions. This is a privileged operation requiring admin credentials. For example, if a new Qari's recitation becomes the reference standard, the admin uploads the audio files and updates metadata.

Manage System Operations: Encompasses system administration tasks such as managing user accounts (e.g., banning abusive users), and configuring AI model parameters (e.g., error detection threshold).

5.2 Class Diagram

The Class Diagram illustrates the object-oriented structure of the system, showing the relationships between key classes including User, Recitation, Verse, and TajweedRule entities.

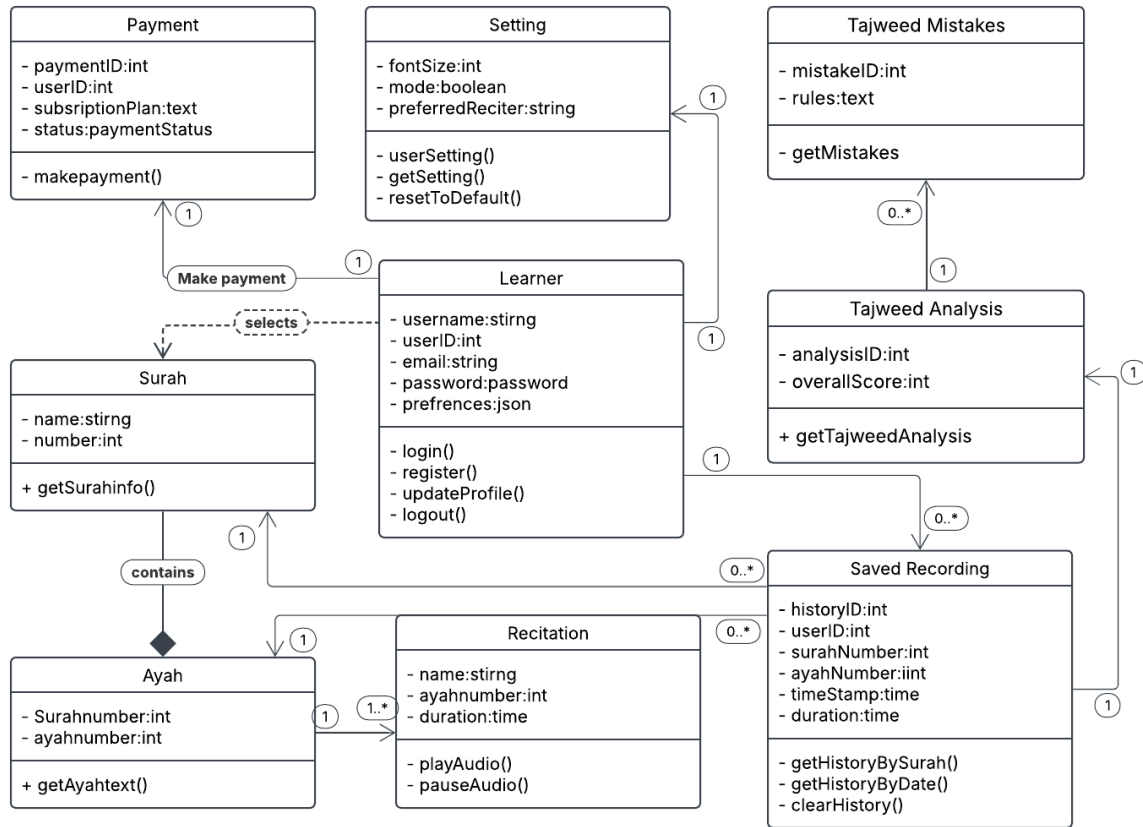


Figure 5.2: Class Diagram

5.2.1 Class Diagram Explanation

This diagram defines the object-oriented architecture with nine interconnected classes that form the backbone of the Mujawwad system.

Learner Class: The Learner class serves as the primary actor with attributes username, userID, email, password, and preferences (JSON format). It provides methods for authentication and profile management.

Surah Class: Represents a Quranic chapter with name (string) and number (integer) attributes. It has a composition relationship (contains) with Ayah, meaning Ayahs cannot exist independently of a Surah.

Ayah Class: Represents individual verses with Surahnumber and ayahnumber as attributes. The getAyahtext() method retrieves the Arabic text of the verse.

Recitation Class: Manages audio playback functionality with attributes name (identifier), ayahnumber (reference), and duration (time). Methods include playAudio() to start playback and pauseAudio() to pause. This class bridges the gap between the Quranic text (Ayah) and user recordings (Saved Recording).

Saved Recording Class: Stores historical recitation data with historyID (primary key), userID (foreign key), surahNumber, ayahNumber, timeStamp (when recorded), and duration.

Tajweed Analysis Class: Contains AI-generated evaluation results with analysisID

and overallScore. The getTajweedAnalysis() method retrieves the complete analysis.

Tajweed Mistakes Class: Stores individual error details with mistakeID and rules. The getMistakes() method returns all errors found in a specific analysis.

Setting Class: Manages user interface preferences including fontSize, mode (dark/light theme), and preferredReciter.

Payment Class: Handles subscription management for the user where user can pay for the premium features of the app (e.g. advance analysis, saved recordings for 1 month or so).

5.3 Sequence Diagram

The Sequence Diagram demonstrates the detailed message flow and interactions between the Learner, Frontend, Backend, and Database components throughout the recitation analysis workflow, including authentication, verse retrieval, audio processing, and result delivery.

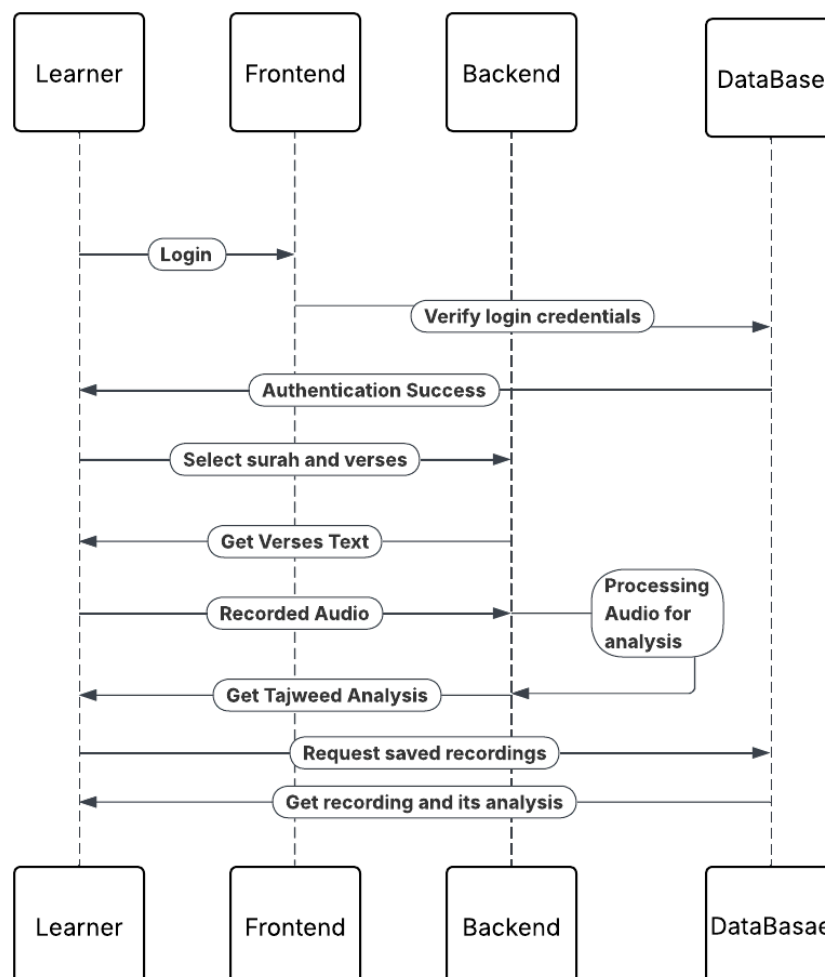


Figure 5.3: Sequence Diagram

5.3.1 Sequence Diagram Explanation

This diagram illustrates the chronological interaction between four key components—Learner, Frontend, Backend, and Database—across a complete user session.

Phase 1: Authentication (Steps 1-6): The Learner initiates a "Login" action on the Frontend. The Frontend sends a "Verify login credentials" HTTP request to the Backend API server. The Backend queries the Database to validate username and password. If credentials match, the Database returns user profile data to the Backend. The Backend generates a JWT authentication token and sends an "Authentication Success" response back to the Frontend. Finally, the Frontend displays the authenticated home screen to the Learner.

Phase 2: Verse Selection (Steps 7-12): The Learner performs a this action, choosing which Quranic passage to recite from the drop down. The Frontend sends this selection to the Backend via an API call. The Backend queries the Database to retrieve the Arabic text, phonetic breakdown, and applicable Tajweed rules for the selected verse. The Database returns this data to the Backend, which formats it as JSON. The Backend responds with Text (Ayah) containing the complete verse information.

Phase 3: Recording and AI Analysis (Steps 13-17): The Learner completes their recitation, and the Frontend captures "Recorded Audio" as a WAV (audio) file. The Frontend uploads this audio blob to the Backend via a multipart HTTP POST request. The Backend receives the file and start processing audio for analysis. This internal process involves the AI Engine extracting phonemes, the Tajweed Comparator aligning them with the reference, and the Error Classifier identifying mistakes. Once complete, the Backend sends results.

Phase 4: History Retrieval (Steps 18-23): Learner can review past performance. The Frontend sends a query to the Backend with optional filters (by Surah or date). The Backend queries the Database, which returns a list of recordings with metadata. The Backend responds with the JSON array.

5.4 Logical E/R Diagram

The Entity Relationship Diagram illustrates the database schema showing the relationships between entities such as User, Recitation, Verse, Surah, and TajweedError with their respective attributes and cardinalities.

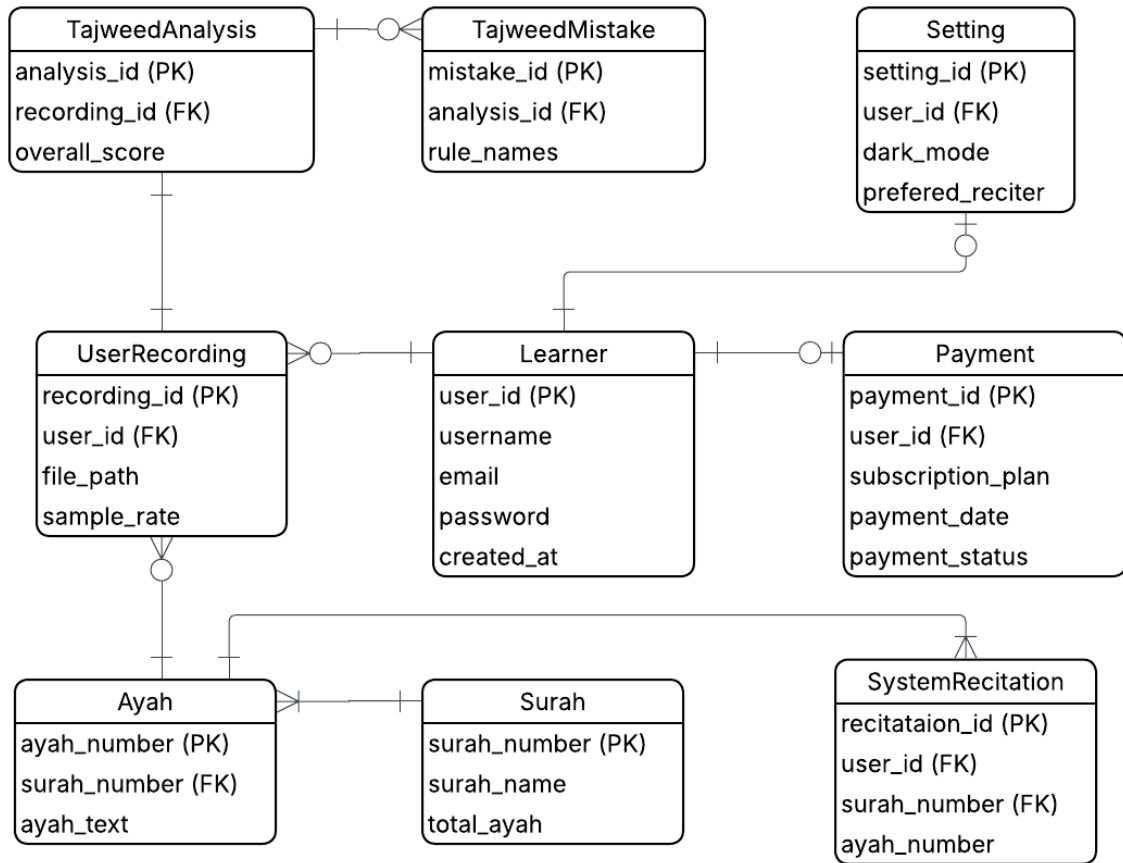


Figure 5.4: E/R Diagram

5.4.1 Entity-Relationship Diagram Explanation

This diagram defines the relational database schema with nine entities and their foreign key relationships, primary keys, and cardinalities.

Learner Entity: Contains columns `user_id`, `username`, `email`, `password`, and `created_at`. The `user_id` serves as the primary key and is referenced as a foreign key in `UserRecording`, `Payment`, `Setting`, and `SystemRecitation` tables.

UserRecording Entity: Stores metadata about audio files with `recording_id` (PK), `user_id`, `file_path`, and `sample_rate`. It has a many-to-one relationship with `Ayah`, as users can record the same verse multiple times.

TajweedAnalysis Entity: Contains `analysis_id` (PK), `recording_id` (FK), and `overall_score`. The one-to-one relationship with `UserRecording` is enforced by a unique constraint on `recording_id`. The one-to-many relationship with `TajweedMistake` allows multiple errors per analysis.

TajweedMistake Entity: Stores individual errors with `mistake_id` (PK), `analysis_id` (FK), and `rule_names` (text array or JSON listing violated rules like "Madd," "Ghun-nah"). The many-to-one relationship with `TajweedAnalysis` means multiple mistakes can belong to one analysis session.

Setting Entity: User preferences table with `setting_id` (PK), `user_id` (FK), `dark_mode`,

and preferred_reciter. The one-to-one relationship with Learner is implemented through a unique foreign key constraint on user_id.

Payment Entity: Subscription tracking with payment_id (PK), user_id (FK unique), subscription_plan, payment_date (timestamp), and payment_status. The one-to-one relationship with Learner ensures each user has one active payment record.

Surah Entity: Quran chapter metadata with surah_number (PK, integer 1-114), surah_name (string in Arabic), and total_ayah (integer count of verses). The one-to-many relationship with Ayah means each Surah contains multiple verses. The surah_number is referenced by Ayah as a foreign key.

Ayah Entity: Verse data with ayah_number (PK), surah_number (FK), and ayah_text. The composite key (surah_number, ayah_number) uniquely identifies each verse. The many-to-one relationship with Surah is enforced by the foreign key constraint.

SystemRecitation Entity: Tracks user practice history with recitation_id (PK), user_id (FK), surah_number (FK), and ayah_number. This entity implements a many-to-many relationship between Learner and Ayah, recording which verses each user has practiced. The triangle symbol indicates this is a weak entity dependent on Learner, Surah, and Ayah.

5.5 State Machine Diagram

The State Machine Diagram represents the various states and transitions within the authentication and recording modules, capturing the lifecycle of a user session from login through successful recitation submission and Tajweed analysis completion.

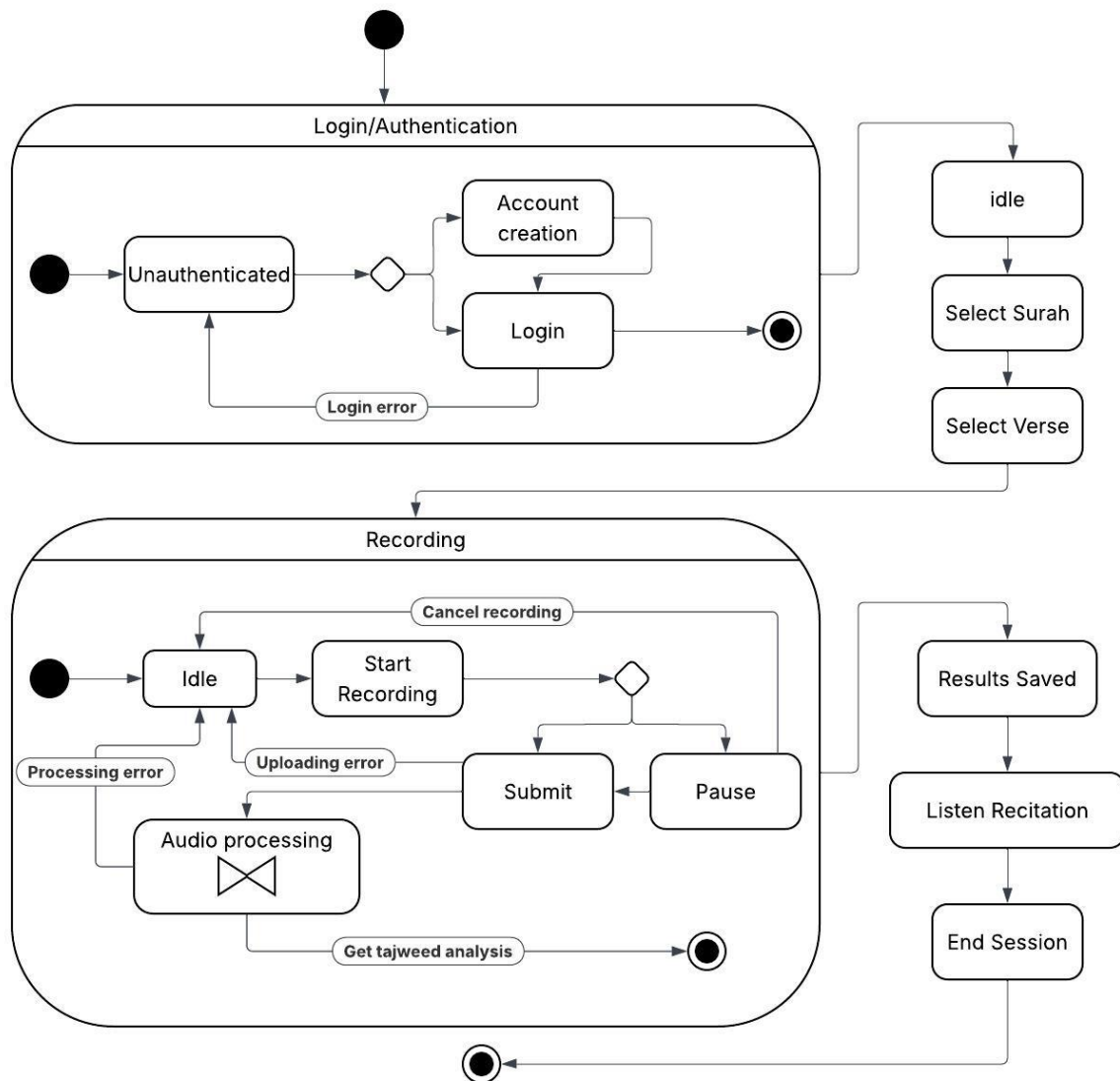


Figure 5.5: State Machine Diagram

5.5.1 State Machine Diagram Explanation

This diagram models the behavioral states of the application using two concurrent state machines: Login/Authentication and Recording.

5.5.1.1 Login/Authentication State Machine

Initial State (Black Circle): The system starts when the application launches. The initial transition leads to the Unauthenticated state.

Unauthenticated State: The user is not logged in and cannot access core features. From this state, a decision point (diamond) offers two paths. Path A leads to "Account creation" where new users register by providing username, email, and password. After successful registration, the system returns to Unauthenticated because the user must explicitly log in after creating an account—registration does not auto-authenticate. Path B leads to the Login state.

B leads directly to the "Login" state.

Login State: The user enters credentials. Two outcomes are possible. On success, the system transitions to the Final State. On failure (invalid credentials), a "Login error" transition loops back to Unauthenticated, allowing the user to retry.

5.5.1.2 Post-Authentication States (Right Side)

Idle State: The authenticated user is on the home screen, not actively performing any action. This is the entry point after login completion.

Select Surah State: The user browses a list of Surahs and taps one. The system loads the chapter metadata (name, number of verses).

Select Verse State: After chosen Surah, the user picks one or more Ayahs to recite. After selection, the system transitions to the Recording state machine.

5.5.1.3 Recording State Machine

Initial State: Entry point for the recording module.

Idle State: The recording interface is displayed with a large microphone button. The verse text is shown on screen. From Idle, the user taps to transition to "Start Recording."

Start Recording State: The device microphone is activated, capturing audio at 16kHz sample rate. A decision point (diamond) offers three paths, Submit (when recording is complete), Pause (temporary stop), or Cancel recording.

Pause State: Audio capture is temporarily suspended. The user can review what they've recorded so far or prepare to continue. From Pause, the only forward path is Submit—the user cannot resume recording in this simplified model.

Submit Transition: The audio buffer is packaged as a WAV file and uploaded to the server.

Audio Processing State (Hourglass Symbol): This represents an asynchronous background operation on the server. The model extracts phonemes, the Tajweed Comparator aligns them, and the Error Classifier detect errors. Three transitions are possible from this state. On success, leads to the Final State,. On failure, two error paths exist, "Processing error" (Backend failure) or "Uploading error".

5.5.1.4 Post-Processing States (Right Side)

Results Saved State: The analysis data will save in database.

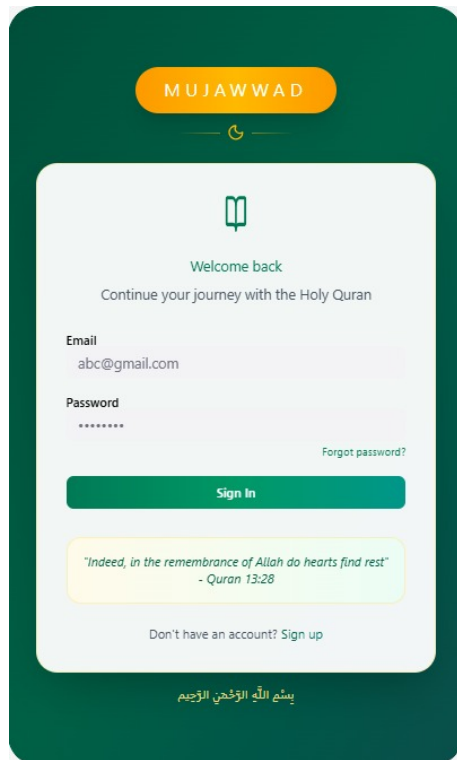
Listen Recitation State: The user can replay their audio to compare against the reference recitation. The UI highlights errors in the verse text.

End Session State: The user closes the results screen. The system transitions back to the global Start state (bottom bullseye), allowing another recording session to begin.

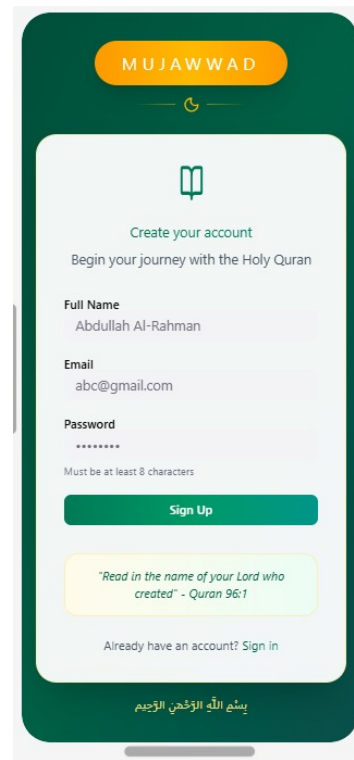
5.6 Graphical User Interface

This section presents the user interface mockups for the mobile application, demonstrating the design and flow of key screens.

5.6.1 Authentication Screens



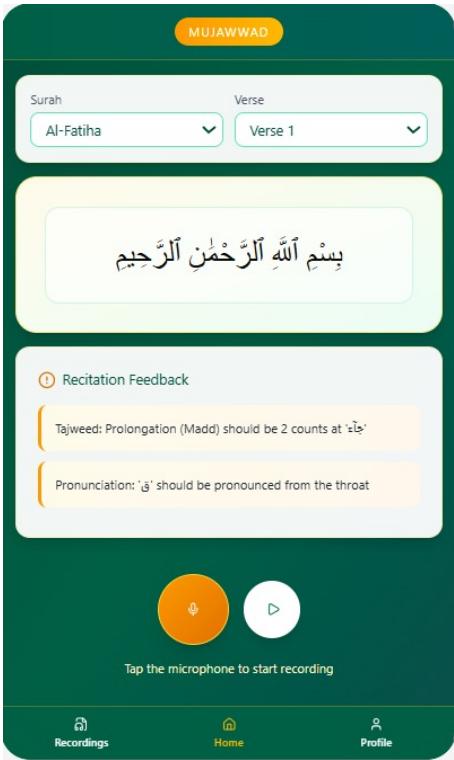
(a) Login Screen



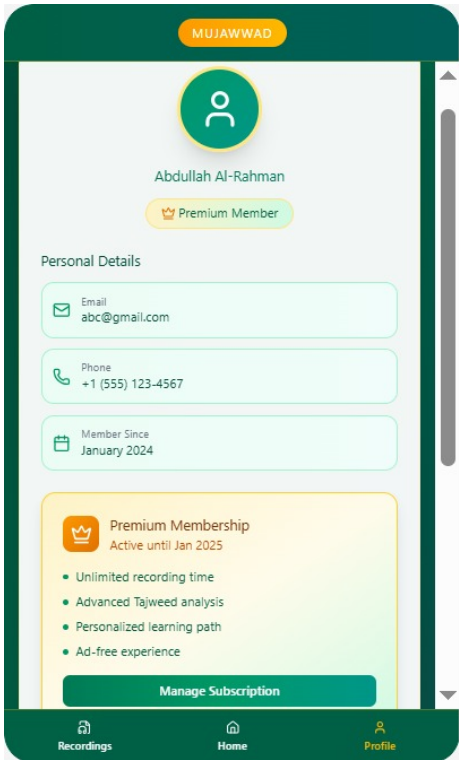
(b) Account Creation Screen

Figure 5.6: Authentication Interface

5.6.2 Main Application Screens



(a) Home Screen



(b) Profile Screen

Figure 5.7: Main Application Interface

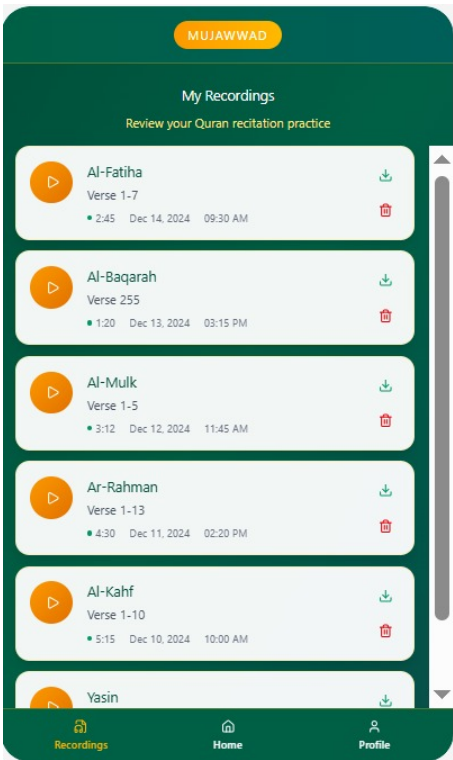


Figure 5.8: Previous Recordings Screen

Bibliography

- [1] C. E. Shannon, “Communication in the presence of noise,” *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, Jan. 1949, doi: 10.1109/jrproc.1949.232969.
- [2] “Django REST Framework,” Django Software Foundation. [Online]. Available: <https://www.django-rest-framework.org/>.
- [3] “Introduction to JSON Web Tokens,” Auth0. [Online]. Available: <https://jwt.io/introduction>.